Puppet Data-Centre Automation Solution

Managing data centres comprising hundreds of systems is tough. And when the machines run on *NIX (Linux and other UNIX-like OSs), the job gets tougher. Fortunately, configuration-management systems help bring things under control, by automating common configuration tasks, making systems manageable from a central location. This article focuses on setting up a prominent FOSS configuration-management system named Puppet.

A systems administrator’s job must include automation and simplification of day-to-day tasks, and more, so that there is time to tackle new projects and assignments. When dealing with hundreds of servers (or even when it’s just a little more than two machines), it becomes tedious managing all the files, users, groups and the systems. It gets even worse if we have multiple operating systems in use in a single data centre/location. Windows eases the burden with Active Directory and group policies, but it isn’t that easy with UNIX machines. The Free Software directory services available at present can, at the most, help manage users, groups and machines, but not files, services, packages—or even support for monitoring.

Here’s where configuration-management engines come to our rescue. Some of those available in the market are CFEngine, Puppet, Bcfg2, Etch, Pcfengine and FusionInventory. The top contenders are CFEngine and Puppet, with support for all OSs (development for Puppet’s support for Windows is in progress—under *References*, see #1). CFEngine might fare better than Puppet in many areas, except for the fact that the Nova module of CFEngine is not available for free download, while Puppet is completely free software, downloadable and distributable under the terms of the GPL. Besides, Puppet’s abstraction layer concept is also a strong feature. Let us, therefore, look at it in detail.

Puppet features

- A full-fledged client/server model.
- It is written completely in Ruby.
- A platform-independent declarative language for configuration.
- A platform-dependent Resource Abstraction Layer. For example, *User* is the configuration syntax used for *useradd* in GNU/Linux, and *adduser* in FreeBSD.
- It depends on an independent Ruby package called *Facter*, which is a full-fledged hardware knowledge-base.